Artificial Intellegence

Python project Report

Project number :12

Prepared by :
Nervana Mohamed
Mariam Attia
Mariam Ibrahim
Mostafa Emad
Hager Rezk

Table Of Contents

1 introduction

1.1 project Overview

1.2 Objectives

2 Background and Motivation

2.1 What is the Flood-Fill Algorithm?

2.2 Use Cases in Real Life

2.3 Why We Choose This Project

3 System Design

3.1 Grid and Color Representation

3.2 User Interface Design using Tkinter

3.3 Algorithm Selection (BFS vs DFS)

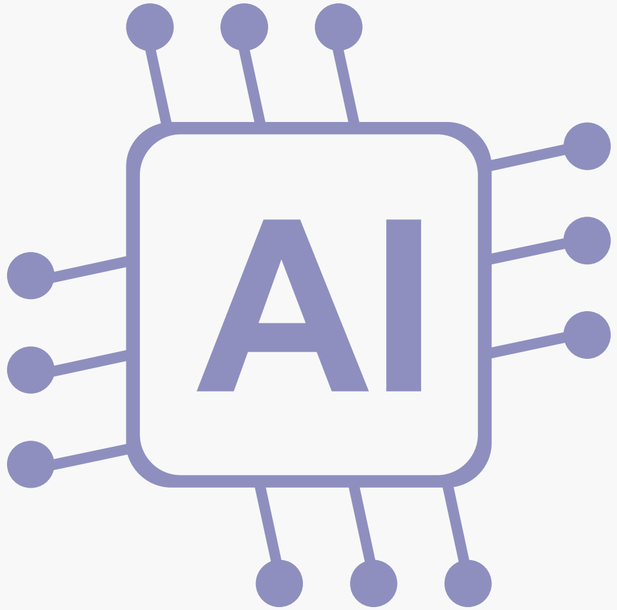
4. Implementation Details

4.1 Code Structure

4.2 BFS Algorithm Logic

4.3 DFS Algorithm Logic

4.4 Drawing and Animation Logic11:18 PM



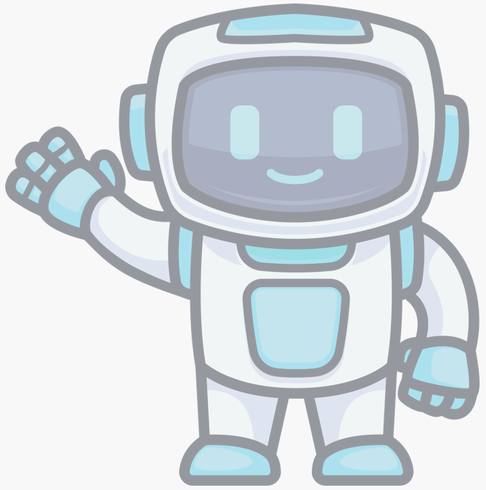
Project Team Members

Name	Group ID	Role
Nervana Mohamed	2401244138	Code organization and led the documentation process including writing the project report and presenting the overall structure of the solution.
Mariam Attia	2401241486	Developed the Breadth-First Search (BFS) algorithm
Mariam Ibrahim	2401246702	Implementing the Depth-First Search (DFS)
Mostafa Emad	2401242497	Collaboratively designed and implemented the Graphical User Interface (GUI) using Tkinter
Hager Rezk	2401245176	

introduction

The Flood-Fill Game is a Python-based project that visualizes how search algorithms like BFS and DFS can solve a color-matching puzzle. Using the Tkinter library, we built an interactive grid where the goal is to fill all cells with one color. The project demonstrates how basic AI techniques can be applied in a fun and visual way, combining logic, design, and problem-solving skills.

What makes this project unique is its ability to turn abstract algorithmic concepts into a hands-on, visual simulation that’s both fun and intellectually engaging. Instead of dealing with numbers or graphs, users observe how the algorithm “thinks,” how it chooses the next move, and how different strategies affect the outcome.



CASE STUDY



This project serves as a practical case study on how classical search algorithms can be applied to solve decision-based problems in a dynamic, visual environment. The core idea revolves around a flood-fill puzzle where the algorithm must determine the most efficient sequence of color changes to make the entire grid uniform. This mirrors real-world challenges such as pathfinding in robotics, image segmentation, and territory control in strategy games.

The graphical interface provided an intuitive way to visualize each step taken by the algorithms, making it easier to analyze efficiency, performance, and logic flow. Through this project, we explored key AI concepts such as **state space traversal**, **heuristic-free decision making**, and **grid-based problem solving**, all within a game-based framework that enhanced understanding and engagement.

This case study demonstrates that even basic AI techniques, when paired with thoughtful design and visualization, can create impactful tools for learning and exploration.

1-Importing Libraries

We import necessary Python libraries:

- tkinter for creating the GUI.
- random to generate a random grid.
- time to introduce delay during the automatic solving animation.

```
import tkinter as tk
import random
import time
```

2-Constants

These constants define:

- The grid size (12x12).
- The size of each cell in pixels.
- The number of different colors.
- A list of colors used for the cells.

python Copy

```
GRID_SIZE = 12
CELL_SIZE = 30
NUM_COLORS = 6
COLORS = ["red", "green", "blue", "yellow", '
```

3-FloodFillGame Class Initialization:

:This is the constructor of the main class. It:

- Initializes the GUI window.
- Creates the grid filled with random color indices.
- Initializes the canvas to draw the grid.
- Adds buttons for BFS, DFS, and resetting the grid.

```
class FloodFillGame:
    def __init__(self, master):
```

4-draw_grid() Method:

This method:

- Clears the canvas.
- Draws each cell as a colored rectangle based on its value from the grid.

```
def draw_grid(self):
```

5. flood_fill_bfs() Method :

This implements the Breadth-First Search (BFS) flood-fill algorithm:

- It changes all connected cells from the starting color to a new color.
- It uses a queue to explore neighbors in BFS order.

```
def flood_fill_bfs(self, srow, scol, newcolor):
```

6. flood_fill_dfs() Method:

This is the Depth-First Search (DFS) version of flood fill:

- It uses a stack to simulate DFS.
- It keeps track of visited cells and updates connected cells with the new color.

```
def flood_fill_dfs(self, grid, start, new_color):
```

7. get_neighbors_color_counts() Method:

This helper method:

- Calculates how many neighboring cells (around the filled region) have each color.
- This is used to decide the next best color for automatic solving.

```
def get_neighbors_color_counts(self):
```

8. is_uniform() Method:

This method checks if all cells in the grid have the same color.

- If so, the puzzle is considered solved.

```
def is_uniform(self):
```

9. auto_solve_bfs() Method:

This function automatically solves the grid using the **BFS flood-fill** approach:

- Repeatedly finds the most frequent neighbor color and applies BFS until the grid is uniform.
- Updates the display after each step with a short delay.

```
def auto_solve_bfs(self):
```

10. auto_solve_dfs() Method:

Similar to auto_solve_bfs(), but uses **DFS** for flood filling.

11. reset_grid() Method:

Resets the grid with a new random set of colors and redraws the canvas.

```
def reset_grid(self):
```

12. Main Execution Block:

This runs the game when the file is executed:

- Creates a Tkinter window.
- Initializes the game.
- Starts the GUI event loop with root.mainloop().

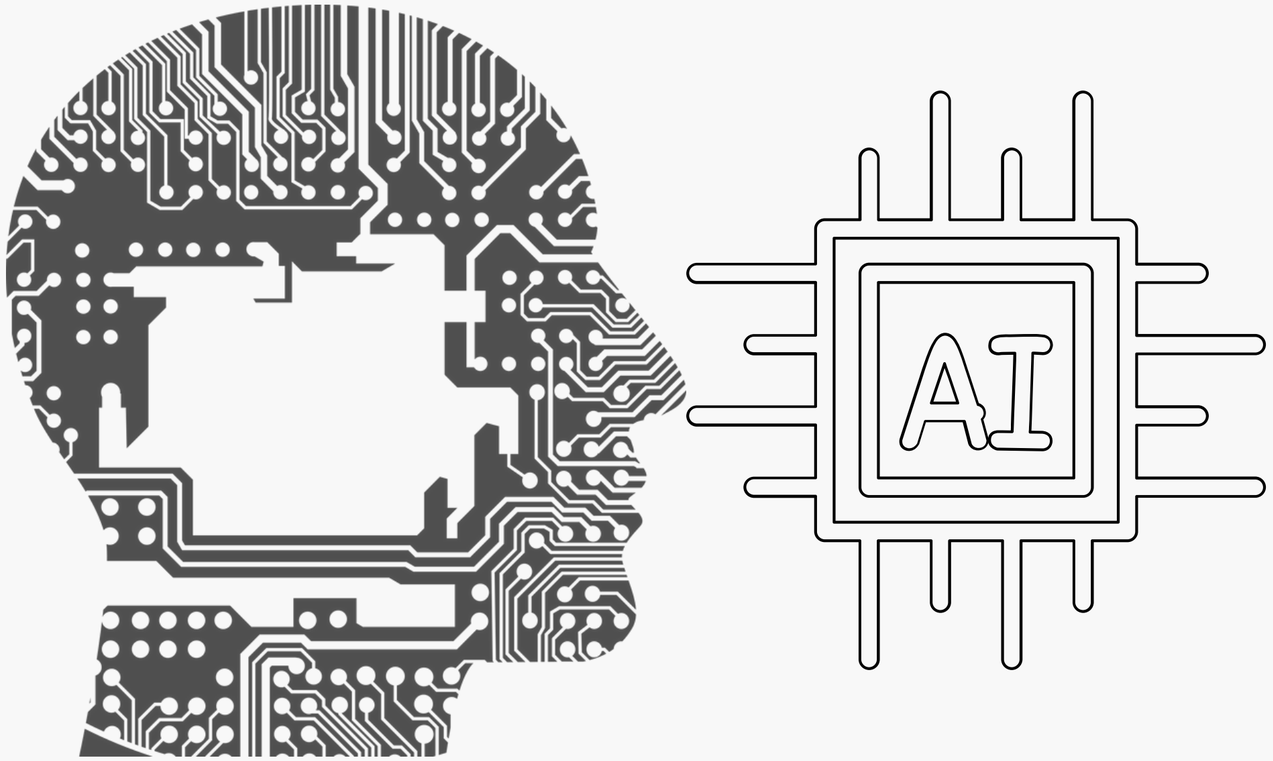
```
if __name__ == "__main__":
```

Code

The draw_grid function is responsible for displaying a grid on a GUI canvas (most likely using Tkinter). It visually represents the internal state of a 2D grid (self.grid) by drawing colored rectangles, where each color corresponds to the value in a specific cell. First, it clears any existing drawings on the canvas using self.canvas.delete("all") to make sure the grid is redrawn cleanly every time this function is called.

Then, it uses two nested for loops to go through every cell in the grid. The outer loop (for i in range(GRID_SIZE)) goes through each row, and the inner loop (for j in range(GRID_SIZE)) goes through each column in that row.

```
def draw_grid(self):
    self.canvas.delete("all")
    for i in range(GRID_SIZE):
        for j in range(GRID_SIZE):
            color = COLORS[self.grid[i][j]]
            x0, y0 = j * CELL_SIZE, i * CELL_SIZE
            x1, y1 = x0 + CELL_SIZE, y0 + CELL_SIZE
            self.canvas.create_rectangle(x0, y0, x1, y1, fill=color, outline="black
```



The auto_solve_bfs method starts by looping continuously while the grid is not uniform, meaning not all cells have the same color. Inside the loop, it calculates the number of neighboring cells for each color using get_neighbors_color_counts(). It then selects the color with the highest count using the max function. Once the best color is chosen, it performs a BFS flood fill starting from the top-left cell (0, 0) using that color, which changes connected areas to match the selected color.

After that, it redraws the updated grid on the canvas with draw_grid(), updates the UI immediately with master.update(), and pauses for 0.3 seconds using time.sleep(0.3) to allow the user to see the changes. The auto_solve_dfs method works in exactly the same way, except it uses DFS (flood_fill_dfs) instead of BFS to apply the color change. Finally, the reset_grid method reinitializes the entire grid by filling it with random colors using nested list comprehensions, and then draws the new random grid using draw_grid().

```
auto_solve_bfs(self):
while not self.is_uniform():
    color_count = self.get_neighbors_color_counts()
    best_color = max(range(NUM_COLORS), key=lambda c: color_count[c])
    self.flood_fill_bfs(0, 0, best_color)
    self.draw_grid()
    self.master.update()
    time.sleep(0.3)

auto_solve_dfs(self):
while not self.is_uniform():
    color_count = self.get_neighbors_color_counts()
    best_color = max(range(NUM_COLORS), key=lambda c: color_count[c])
    self.flood_fill_dfs(self.grid, (0, 0), best_color)
    self.draw_grid()
    self.master.update()
    time.sleep(0.3)

reset_grid(self):
self.grid = [[random.randint(0, NUM_COLORS - 1) for _ in range(GRID_SIZE)] for _ in range(GRID_SI
self.draw_grid()
```

The get_neighbors_color_counts method starts by creating a 2D list called visited to track which grid cells have already been checked. It initializes a queue with the starting cell (0, 0), sets the start_color as the color of that cell, and marks it as visited. It also creates a color_count list initialized with zeros, where each index corresponds to a color and counts how many times it appears around the flood area. Then it enters a while loop that runs as long as there are cells in the queue. In each iteration, it removes the first cell from the queue and checks its four neighbors (up, down, left, right).

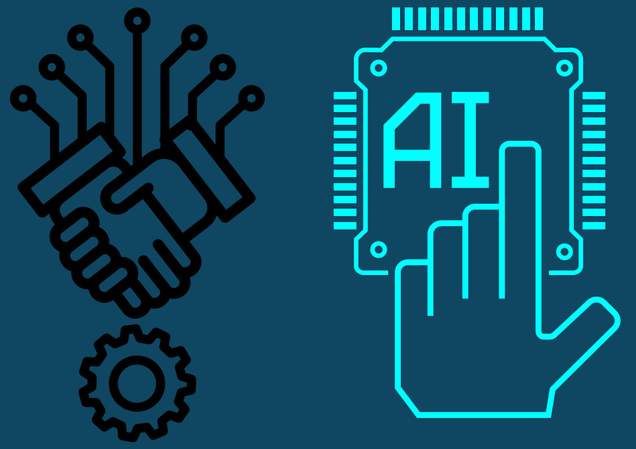


For each neighbor, if it’s within the grid bounds and not visited yet, the code checks its color: if it matches the start color, it’s part of the flood area, so it’s marked as visited and added to the queue. If it’s a different color, it increases the count for that color in color_count. After processing all reachable cells with the same start color, the function returns the color_count list, which now contains the number of neighboring cells for each color adjacent to the flood region.

```
                stack.append((ni, nj))

def get_neighbors_color_counts(self):
    visited = [[False]*GRID_SIZE for _ in range(GRID_SIZE)]
    queue = [(0, 0)]
    start_color = self.grid[0][0]
    visited[0][0] = True
    color_count = [0] * NUM_COLORS
    while len(queue) > 0:
        x, y = queue[0]
        queue = queue[1:]
        for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
            nx, ny = x+dx, y+dy
            if 0 <= nx < GRID_SIZE and 0 <= ny < GRID_SIZE and not visited[nx][ny]:
                if self.grid[nx][ny] == start_color:
                    visited[nx][ny] = True
                    queue.append((nx, ny))
                else:
                    color_count[self.grid[nx][ny]] += 1
    return color_count
```

The flood_fill_bfs method performs a Breadth-First Search (BFS) flood fill starting from a specific cell at (srow, scol) and changes all connected cells with the same original color to a new color (newcolor). It first gets the number of rows and columns in the grid, then stores the color of the starting cell as currentcolor. If the starting cell already has the new color, it returns immediately to avoid unnecessary work.



Otherwise, it initializes a queue with the starting cell and immediately changes its color to newcolor. It defines the four possible directions (up, down, left, right) in which it can move. Then, using a while loop, it processes each cell in the queue: it pops the first cell, checks its four neighbors, and for each neighbor that's within bounds and has the same currentcolor, it updates its color to newcolor and adds it to the queue. This way, all connected cells of the same original color get changed to the new color in a BFS manner.

```
def flood_fill_bfs(self, srow, scol, newcolor):
    rows = len(self.grid)
    cols = len(self.grid[0])
    currentcolor = self.grid[srow][scol]
    if currentcolor == newcolor:
        return

    queue = [(srow, scol)]
    self.grid[srow][scol] = newcolor
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        r, c = queue[0]
        queue = queue[1:]

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and self.grid[nr][nc] == currentcolor:
                self.grid[nr][nc] = newcolor
                queue.append((nr, nc))
```

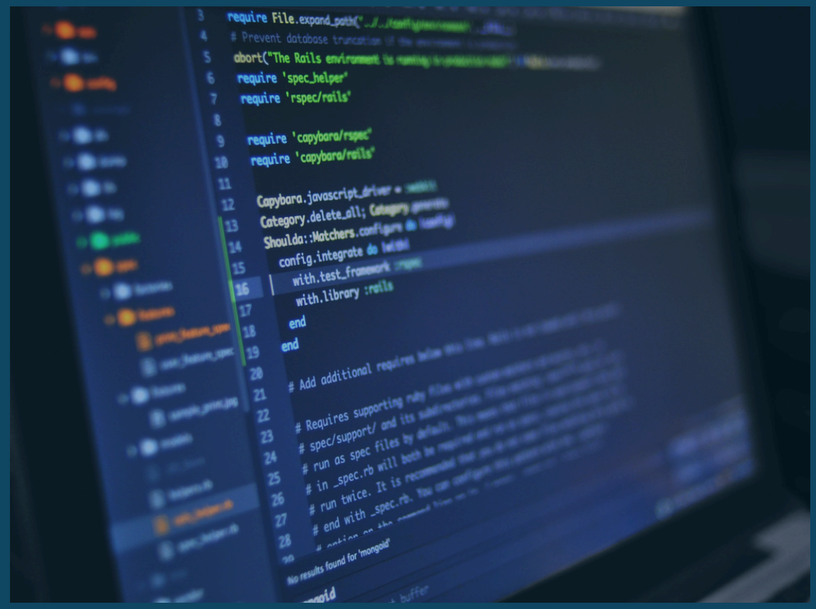
The FloodFillGame class initializes a grid-based color-filling game using Tkinter. In the constructor init, it first saves a reference to the Tkinter root window (master). Then it creates a 2D grid (self.grid) of size GRID_SIZE x GRID_SIZE, where each cell is randomly assigned a color represented by an integer between 0 and NUM_COLORS - 1. Next, it creates a canvas (self.canvas) sized based on the grid and cell size, and displays it using .pack(). It then draws the initial grid by calling self.draw_grid().

```python
def flood_fill_bfs(self, srow, scol, newcolor):
    rows = len(self.grid)
    cols = len(self.grid[0])
    currentcolor = self.grid[srow][scol]
    if currentcolor == newcolor:
        return

    queue = [(srow, scol)]
    self.grid[srow][scol] = newcolor
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        r, c = queue[0]
        queue = queue[1:]

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and self.grid[nr][nc] == currentcolor:
                self.grid[nr][nc] = newcolor
                queue.append((nr, nc))
```



Below that, it creates a separate frame (button_frame) to hold three buttons side by side. It adds a BFS button that triggers self.auto_solve_bfs when clicked, a DFS button that triggers self.auto_solve_dfs, and a Reset button that triggers self.reset_grid. Each button is packed inside the frame with some horizontal padding for spacing. This setup provides a basic UI with a colored grid and controls for solving or resetting it.

The flood_fill_dfs function performs a Depth-First Search (DFS) flood fill on the given grid, starting from the cell start and replacing all connected cells of the same color with new_color. It begins by getting the grid dimensions and extracting the starting coordinates x and y. It saves the original color of the starting cell in original_color.



```
class FloodFillGame:
    def __init__(self, master):
        self.master = master
        self.grid = [[random.randint(0, NUM_COLORS - 1) for _ in range(GRID_SIZE)] for _ in range(GRID_SIZE)]
        self.canvas = tk.Canvas(master, width=GRID_SIZE*CELL_SIZE, height=GRID_SIZE*CELL_SIZE)
        self.canvas.pack()
        self.draw_grid()

        # Frame to hold buttons side by side
        button_frame = tk.Frame(master)
        button_frame.pack(pady=10)

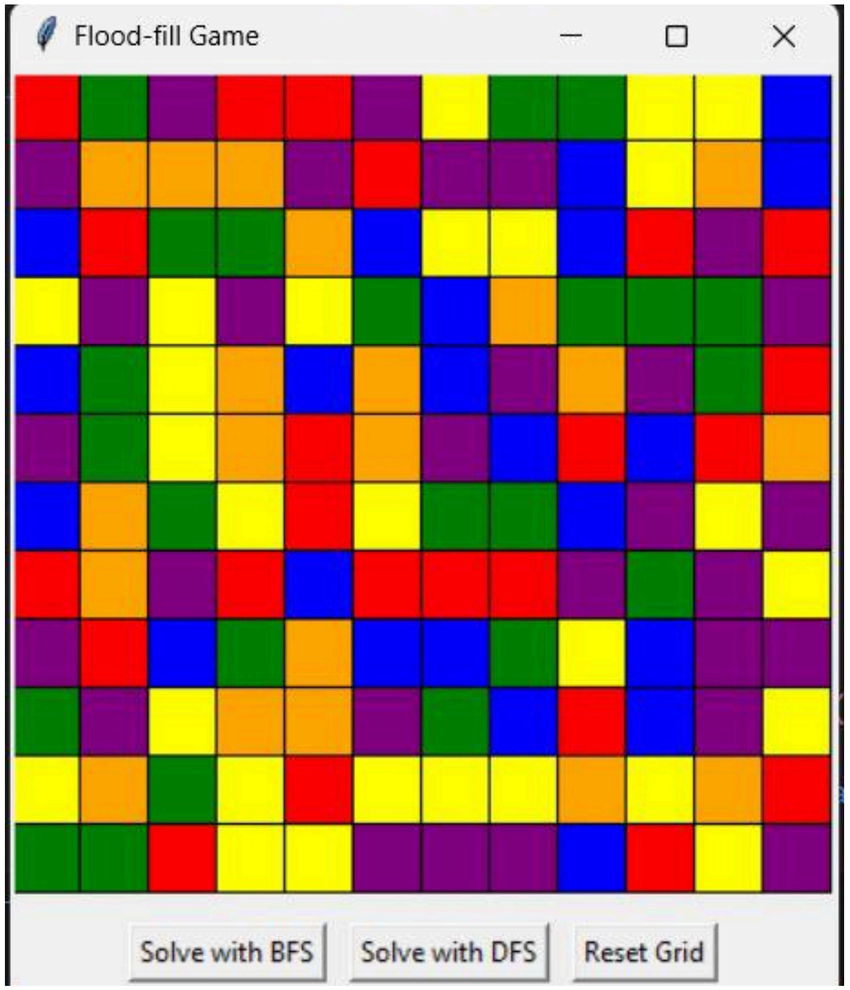
        bfs_btn = tk.Button(button_frame, text="Solve with BFS", command=self.auto_solve_bfs)
        bfs_btn.pack(side="left", padx=5)

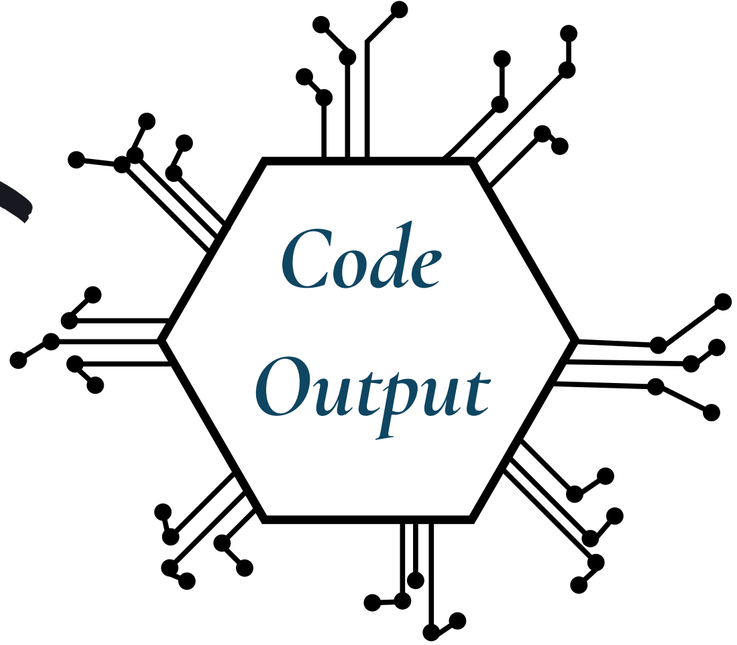
        dfs_btn = tk.Button(button_frame, text="Solve with DFS", command=self.auto_solve_dfs)
        dfs_btn.pack(side="left", padx=5)

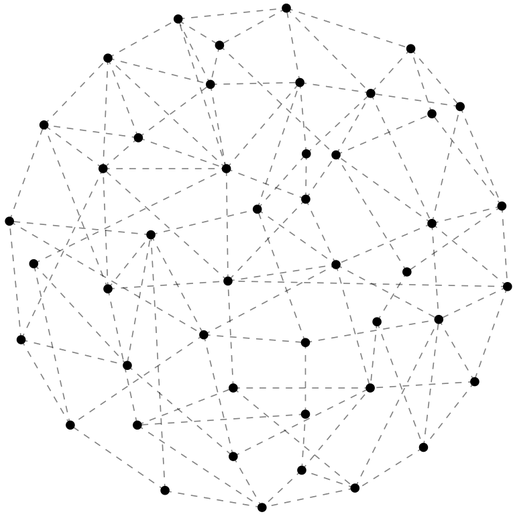
        reset_btn = tk.Button(button_frame, text="Reset Grid", command=self.reset_grid)
        reset_btn.pack(side="left", padx=5)
```

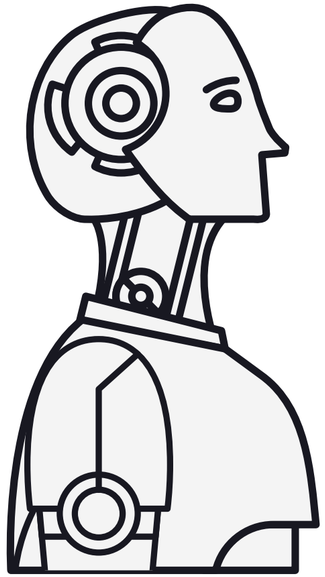
```
def flood_fill_bfs(self, srow, scol, newcolor):
```

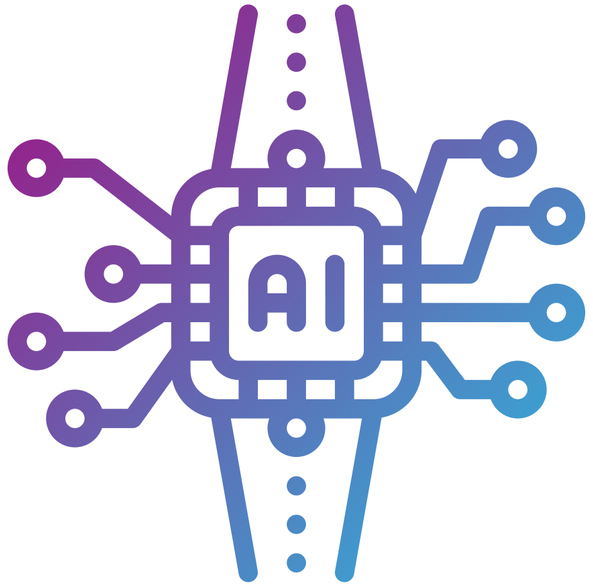
If the new color is the same as the original, it returns immediately to avoid unnecessary work. It initializes a visited set to track visited cells and a stack with the starting position for DFS traversal. Inside the while loop, it pops a cell from the stack and checks if it has already been visited; if so, it skips it. If not, it marks the cell as visited, changes its color to new_color, and then checks its four neighbors (up, down, left, right). For each valid neighbor inside the grid boundaries that has the original color, it adds that cell to the stack for further processing. This continues until all connected cells with the same original color are filled.


Flood-fill Game
Solve with BFS
Solve with DFS
Reset Grid


Code
Output






AI